

Amarcode Staj Raporu - Ekler

İçindekiler

1	Amarcode Staj Raporu — Ekler	1
1.1	Ek A — Proje Bileşenleri ve Depo Yapısı	1
1.2	Ek B — Genel Sistem Mimarisi	2
1.3	Ek C — Temel Çalışma Akışları	2
1.3.1	Ek C.1 — Kullanıcı isteminin yaşam döngüsü	2
1.3.2	Ek C.2 — Önce kaydet, sonra bildir ilkesi	2
1.3.3	Ek C.3 — ACP araç izni ve terminal yönetimi	7
1.4	Ek D — Daemon ve Registry Yaşam Döngüleri	7
1.4.1	Ek D.1 — Daemon durumları	7
1.4.2	Ek D.2 — Güncelleme ve geri alma	9
1.4.3	Ek D.3 — Ajan registry ve kurulum akışı	9
1.5	Ek E — Oturum Yapılandırmasının Eşitlenmesi	9
1.6	Ek F — Seçilmiş Uygulama Ekranları	11
1.6.1	Ek F.1 — Açık ve koyu tema	11
1.6.2	Ek F.2 — Yeni sohbet ekranı	11
1.6.3	Ek F.3 — Dosya ağacı ve kaynak görüntüleyici	11
1.6.4	Ek F.4 — Diff inceleme	11
1.6.5	Ek F.5 — Ajan seçimi	11
1.6.6	Ek F.6 — Model ve yapılandırma seçenekleri	14
1.7	Ek G — İstemci-Daemon RPC Özeti	14
1.8	Ek H — Canlı Olay Türleri	15
1.9	Ek I — Test ve Doğrulama Matrisi	15
1.10	Ek J — Git Tabanlı Geliştirme Özeti	16
1.11	Ek K — Bilinen Sınırlamalar ve Gelecek Çalışmalar	16
1.12	Ek L — İlgili Belgeler	16

1 Amarcode Staj Raporu — Ekler

Bu bölüm, 18 günlük staj raporunda açıklanan çalışmaları destekleyen teknik materyalleri bir araya getirmektedir. Eklerde yer alan mimari açıklamalar, tablolar ve görseller Amarcode kaynak kodu, Git geçmişi, proje belgeleri ve uygulamanın son durumuna ait ekran görüntüleri temel alınarak hazırlanmıştır.

1.1 Ek A — Proje Bileşenleri ve Depo Yapısı

Amarcode tek bir masaüstü arayüzünden oluşmamaktadır. Proje; kullanıcı arayüzü, yerel masaüstü katmanı, bağımsız arka plan servisi, ortak iletişim protokolü, ACP adaptörü ve kayıt hizmetinden meydana gelen bir monorepo yapısına sahiptir.

Bileşen	Konum	Temel sorumluluk
React uygulaması	crates/application/src	Sohbet ekranı, istem girişi, ajan seçimi, ayarlar, dosya ağacı ve diff görüntüleme
Tauri katmanı	crates/application/src-tauri	Masaüstü penceresi, daemon bağlantısı, yerel dosya erişimi ve süreç yönetimi

Bileşen	Konum	Temel sorumluluk
Amarcode daemon	crates/daemon	Kalıcı durum, RPC sunucusu, ACP oturumları, ajan süreçleri ve canlı olaylar
Ortak protokol	crates/protocol	İstemci-daemon sözleşmesi, RPC türleri, olaylar ve TypeScript bağ üretimi
ACP adaptörü	crates/amarcode-acp	OpenAI uyumlu sağlayıcıları ACP standardına bağlama ve çalışma alanı araçları
Daemon registry	crates/daemon-registry	Platforma uygun daemon sürümlerinin ve indirme bilgilerinin yayımlanması
SQLite migration'ları	crates/daemon/migrations	Kalıcı veri şemasının sürümlü olarak oluşturulması ve güncellenmesi
Rapor görselleri	assets/diagrams	Düzenlenebilir Mermaid kaynakları ile SVG ve PNG çıktıları

Rust çalışma alanının ana bileşenleri kök [Cargo.toml](#) dosyasında, JavaScript çalışma alanı ve ortak komutlar ise kök [package.json](#) dosyasında tanımlanmıştır.

1.2 Ek B — Genel Sistem Mimarisi

Ek Şekil B.1 — Kullanıcı, React/Tauri masaüstü uygulaması, bağımsız daemon, SQLite, ACP ajanı, ajan kayıt sistemi ve çalışma alanı arasındaki ilişkiler.

Mimaride masaüstü arayüzü kalıcı verinin sahibi değildir. Kullanıcı komutları Tauri köprüsü üzerinden yerel TCP JSON-line RPC bağlantısına aktarılır. Daemon'ın servis katmanı iş akışlarını yönetir, kalıcı kayıtları SQLite'a yazar ve yalnızca başarılı kayıttan sonra istemcilere olay gönderir. ACP istemcisi kodlama ajanını ayrı bir süreç olarak başlatır ve standart giriş/çıkış üzerinden JSON-RPC iletişimi kurar. Bu ayırım, masaüstü penceresi kapansa bile görevlerin arka planda sürdürülebilmesini sağlar.

Diyagramın düzenlenebilir kaynağı: [system-architecture.mmd](#)

1.3 Ek C — Temel Çalışma Akışları

1.3.1 Ek C.1 — Kullanıcı isteminin yaşam döngüsü

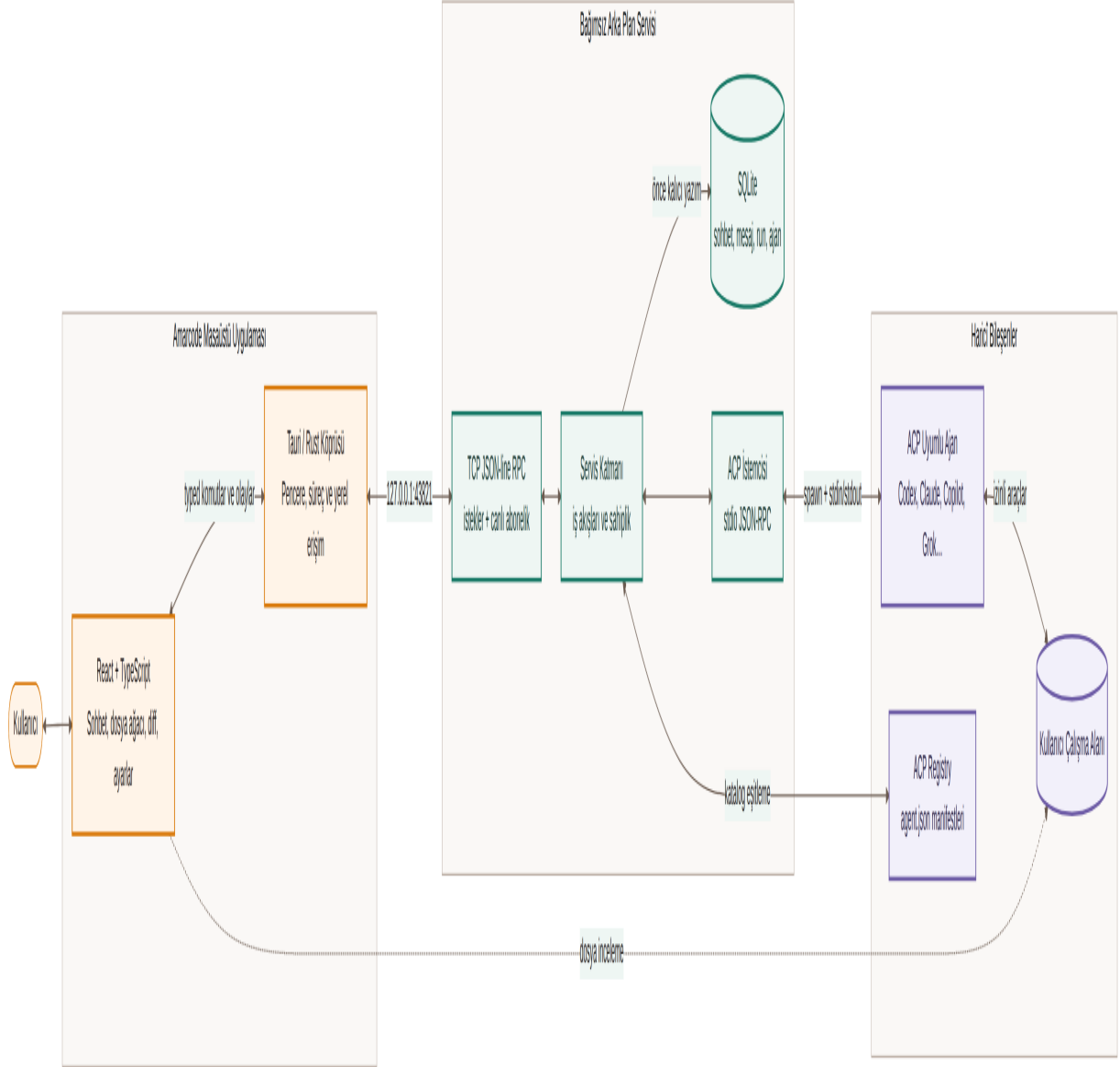
Ek Şekil C.1 — Kullanıcı isteminin React arayüzünden daemon'a ve ACP ajanına ulaşması ile akışlı cevabın arayüze geri dönmesi.

Bu akışta kullanıcı mesajı arayüzde iyimser olarak gösterilir. Daemon önce çalışma ve kullanıcı mesajı kayıtlarını oluşturur, ardından ACP oturumunu başlatır veya sürdürür. Ajan tarafından üretilen her mesaj parçası kalıcı depoya yazıldıktan sonra messagePartAdded olayıyla arayüze bildirilir. Dönüş tamamlandığında mesaj ve çalışma durumu güncellenerek turnUpdated olayı yayımlanır.

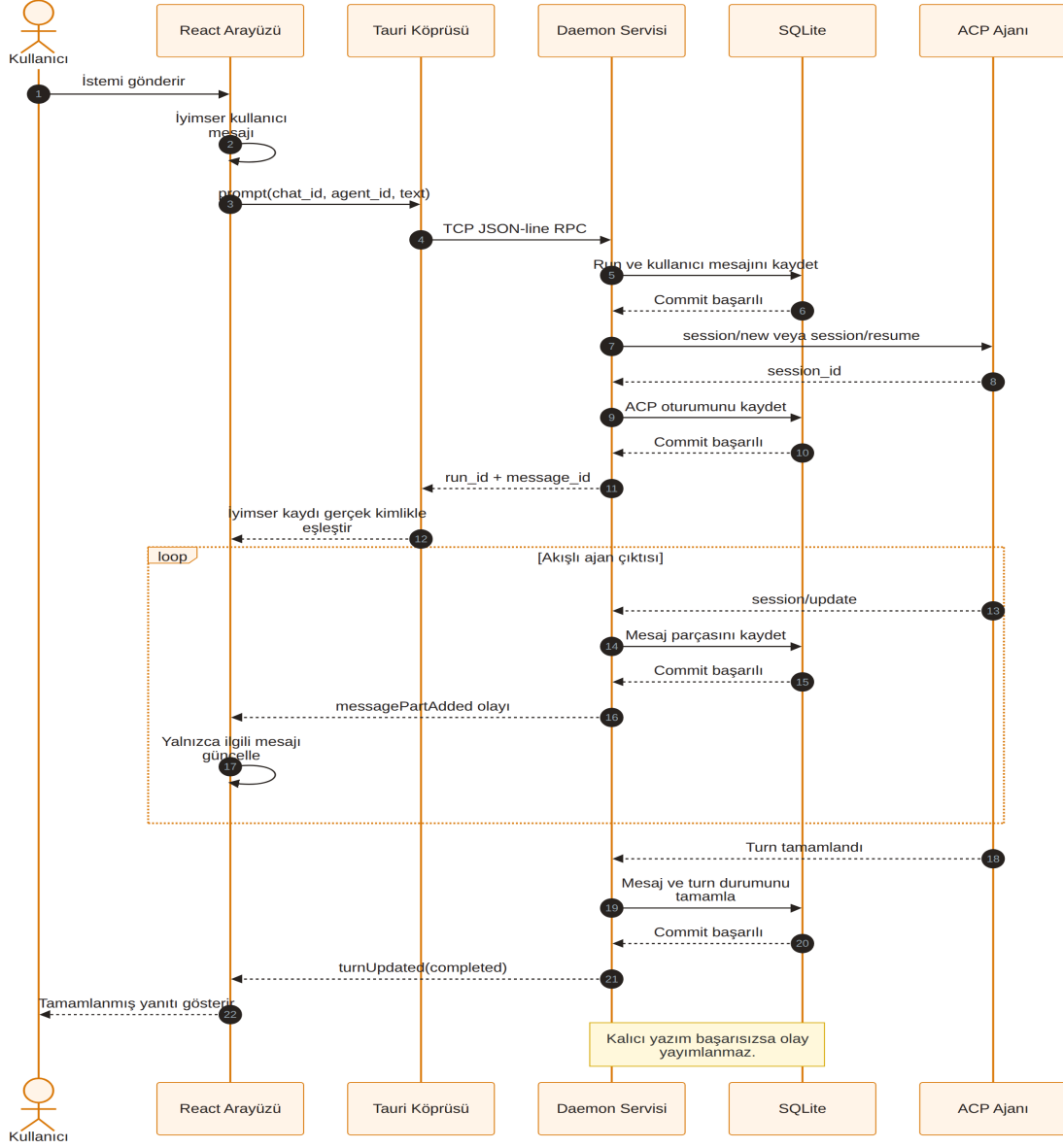
1.3.2 Ek C.2 — Önce kaydet, sonra bildir ilkesi

Ek Şekil C.2 — Bir ACP sinyalinin sahiplik kontrolünden, SQLite işleminden ve olay yayınından geçişi.

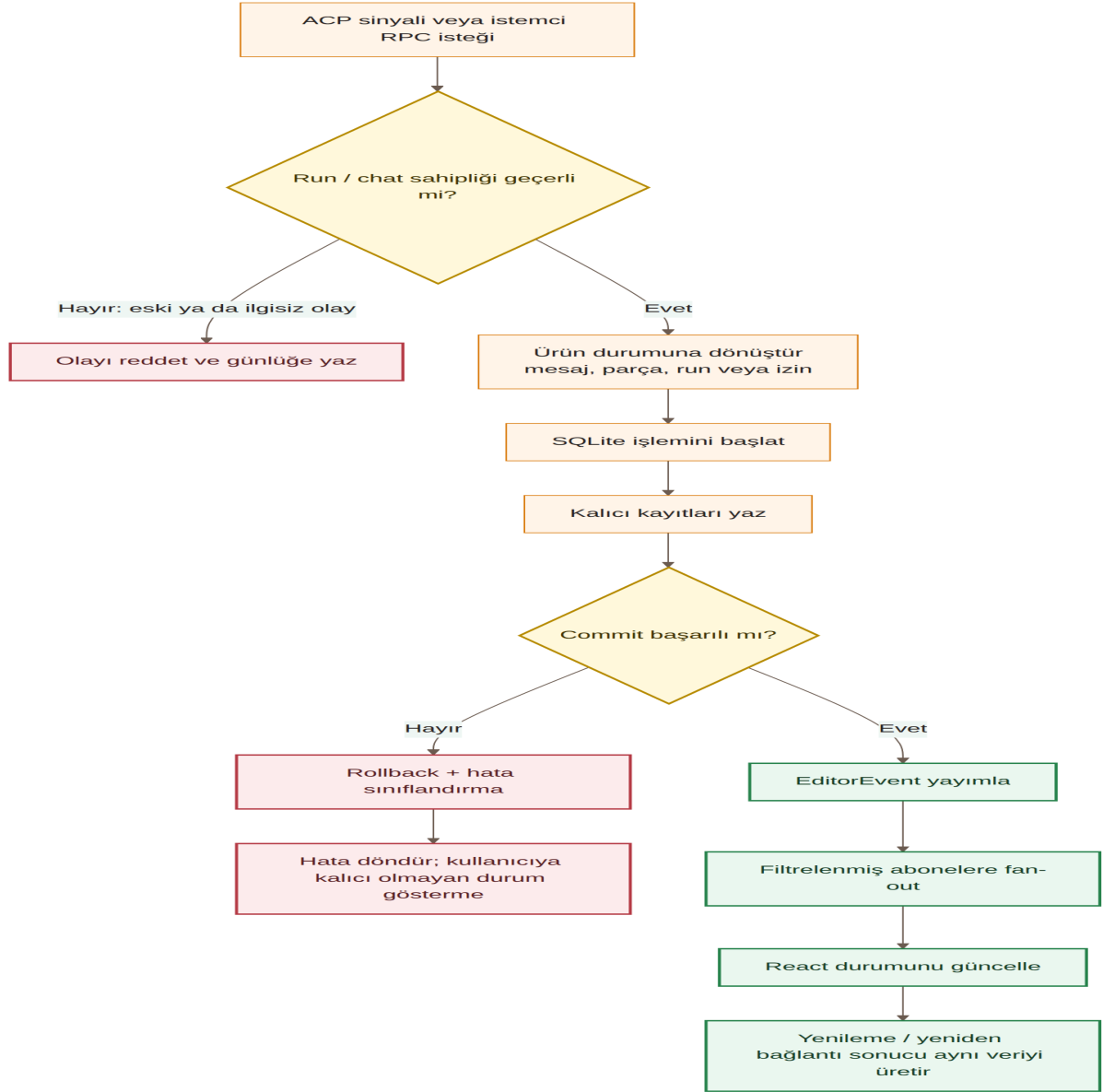
Bu ilke, arayüzün veritabanında bulunmayan bir durumu görmesini engeller. Kalıcı yazım başarısız olursa işlem geri alınır ve başarılı bir ürün olayı yayımlanmaz. Uygulama yenilendiğinde veya bağlantı tekrar kurulduğunda arayüz aynı kalıcı kayıtlardan yeniden oluşturulabilir.



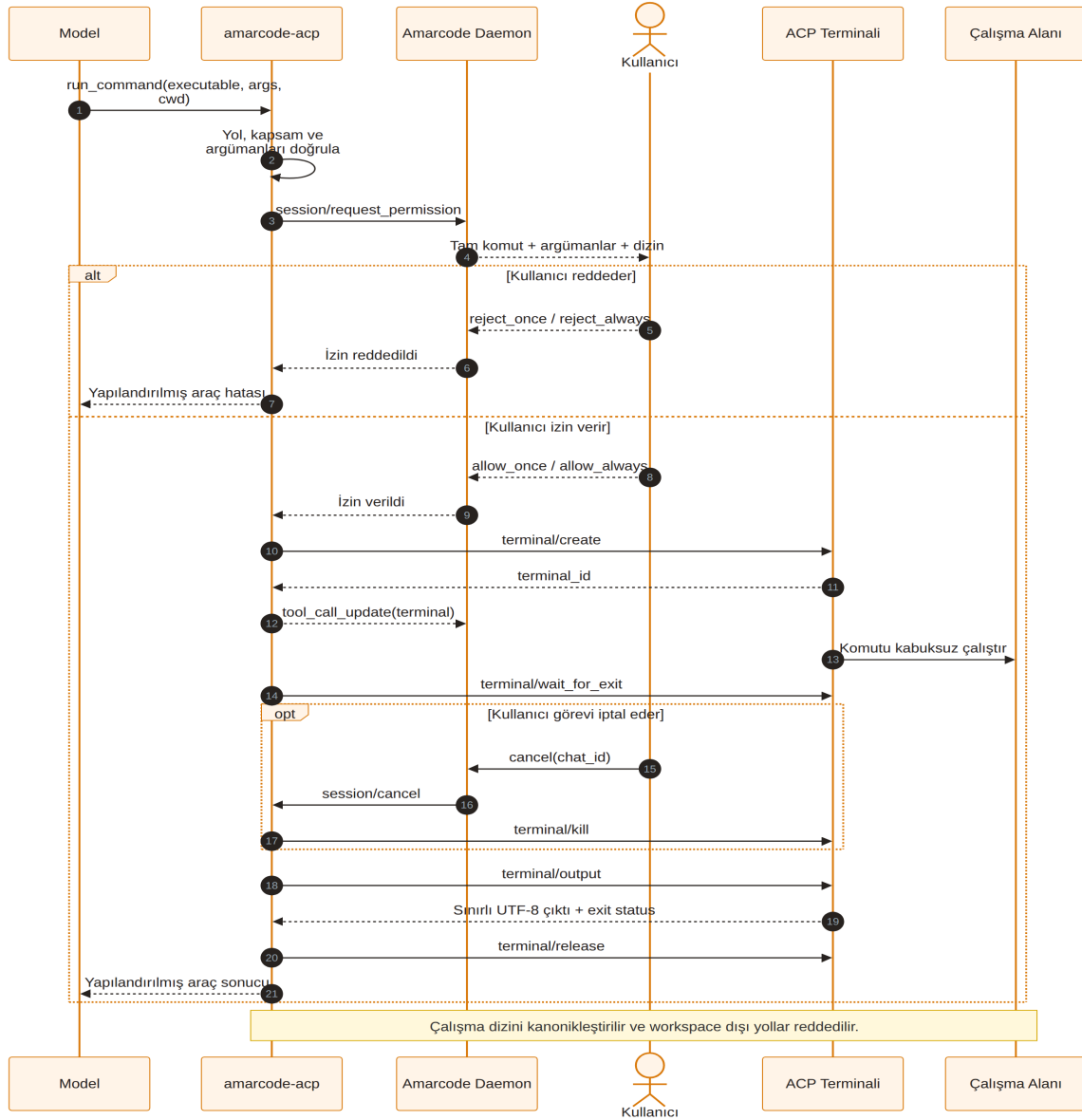
Şekil 1: Amarcode genel sistem mimarisi



Şekil 2: Kullanıcı istemi yaşam döngüsü



Şekil 3: Önce kaydet olay akışı



Şekil 4: ACP araç izni ve terminal sıralaması

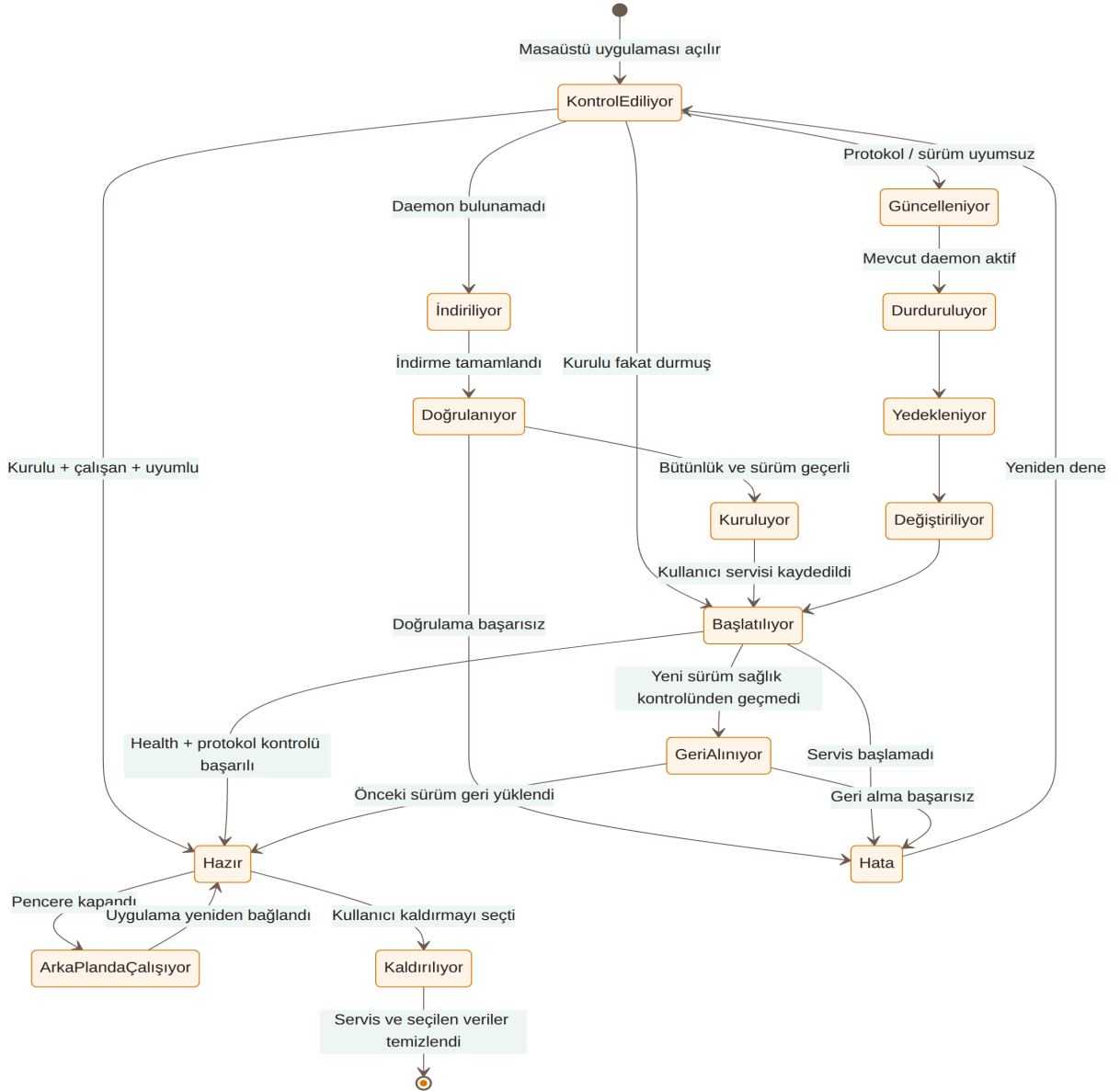
1.3.3 Ek C.3 — ACP araç izni ve terminal yönetimi

Ek Şekil C.3 — Modelin komut çağırısı, kullanıcı izni, terminal oluşturma, çıktı alma, iptal ve kaynak temizleme sırası.

Komutlar örtük bir kabuk üzerinden yorumlanmaz; çalıştırılabilir dosya ve argümanlar ayrı tutulur. Çalışma dizini aktif proje klasörüyle sınırlandırılır. Kullanıcı tarafından verilen izin tam komut, argüman listesi ve çalışma dizini için geçerlidir. İşlem başarıyla tamamlansa, hata verse veya iptal edilse bile terminal kaynağı serbest bırakılır.

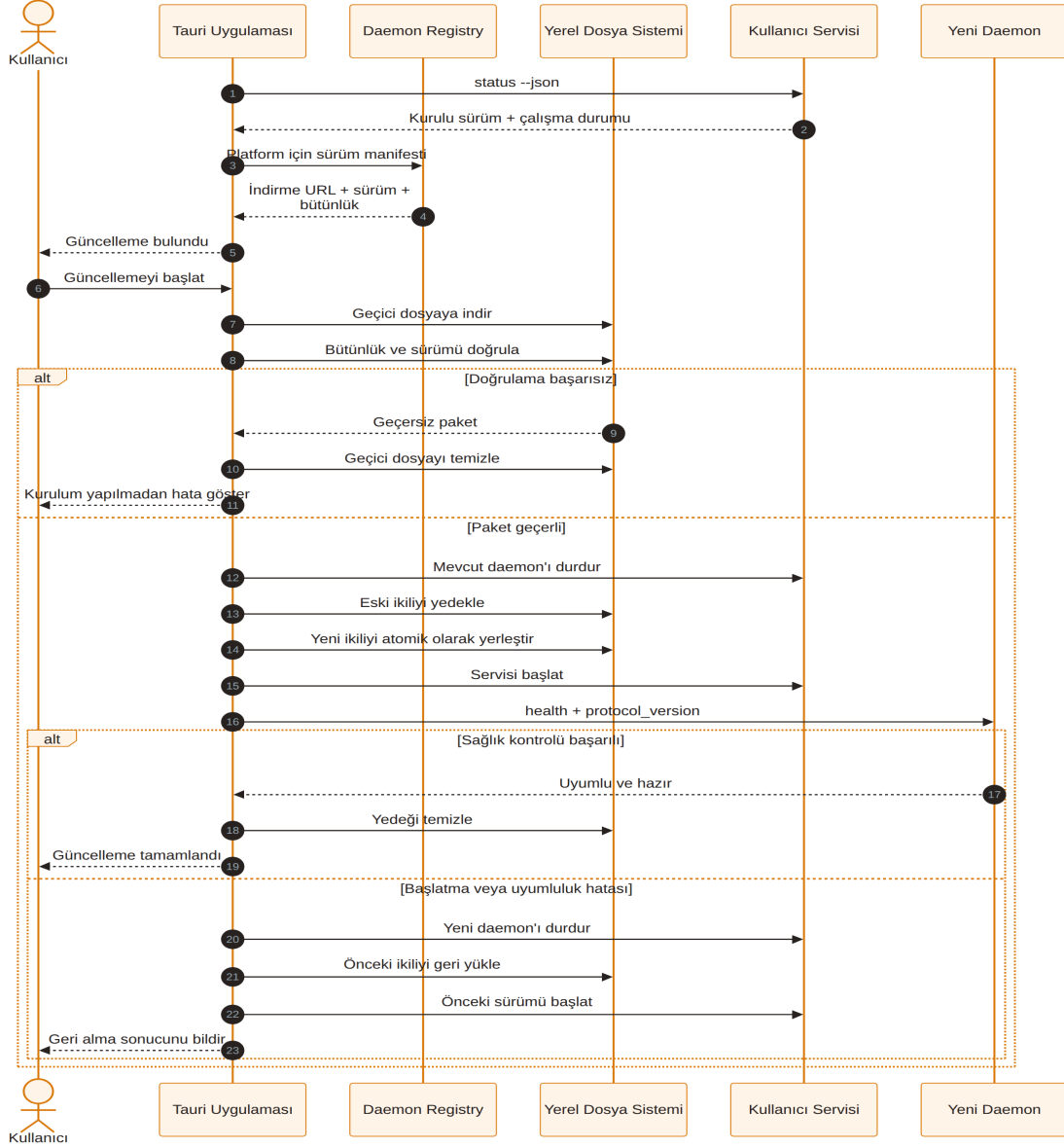
1.4 Ek D — Daemon ve Registry Yaşam Döngüleri

1.4.1 Ek D.1 — Daemon durumları



Şekil 5: Daemon yaşam döngüsü

Ek Şekil D.1 — Daemon kontrolü, indirme, doğrulama, kurulum, başlatma, güncelleme, geri alma ve kaldırma durumları.

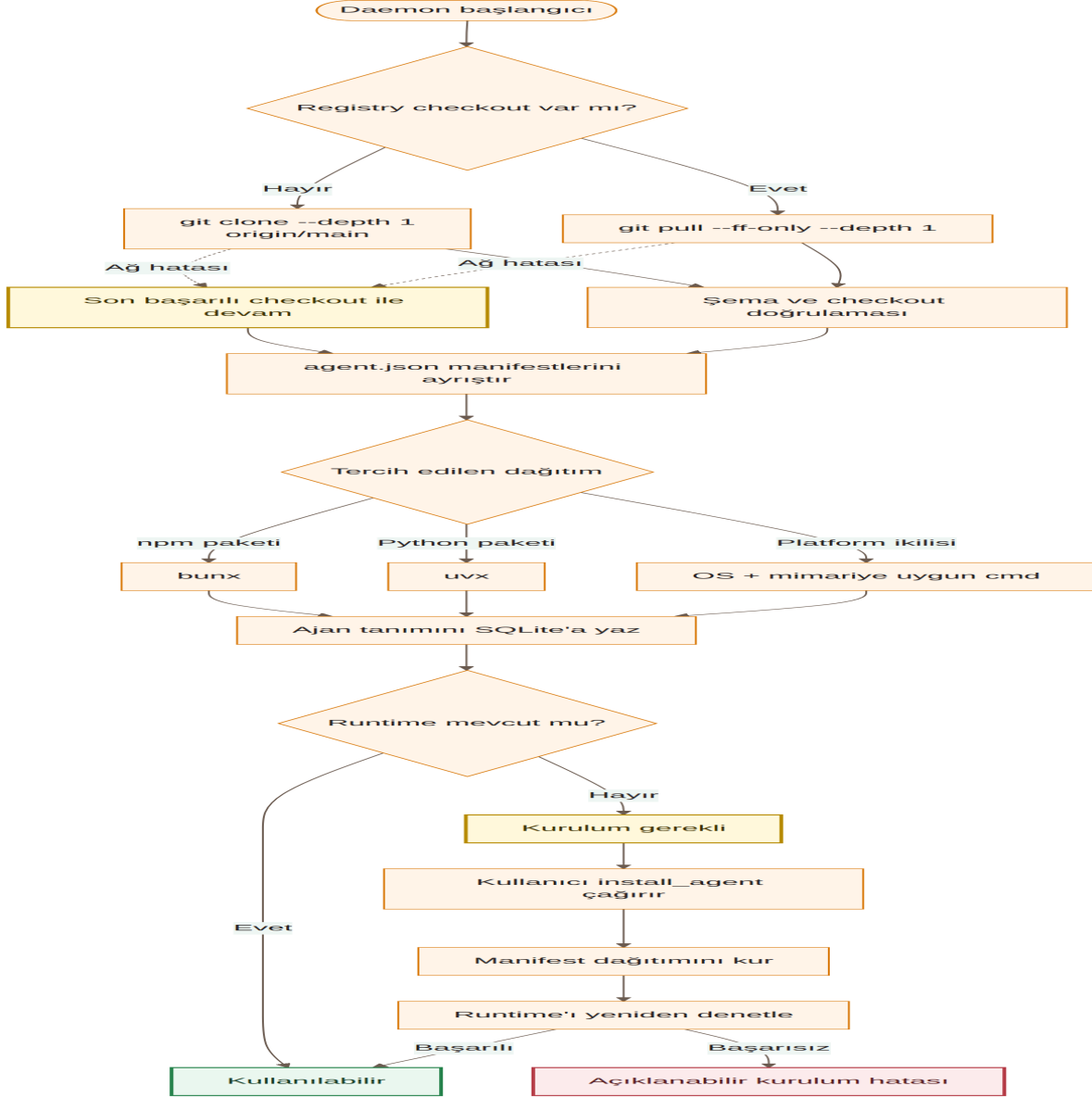


Şekil 6: Daemon güncelleme sıralaması

1.4.2 Ek D.2 — Güncelleme ve geri alma

Ek Şekil D.2 — Yeni daemon sürümünün geçici dosyaya indirilmesi, doğrulanması, yerleştirilmesi ve sağlık kontrolü başarısızsa önceki sürümün geri yüklenmesi.

1.4.3 Ek D.3 — Ajan registry ve kurulum akışı



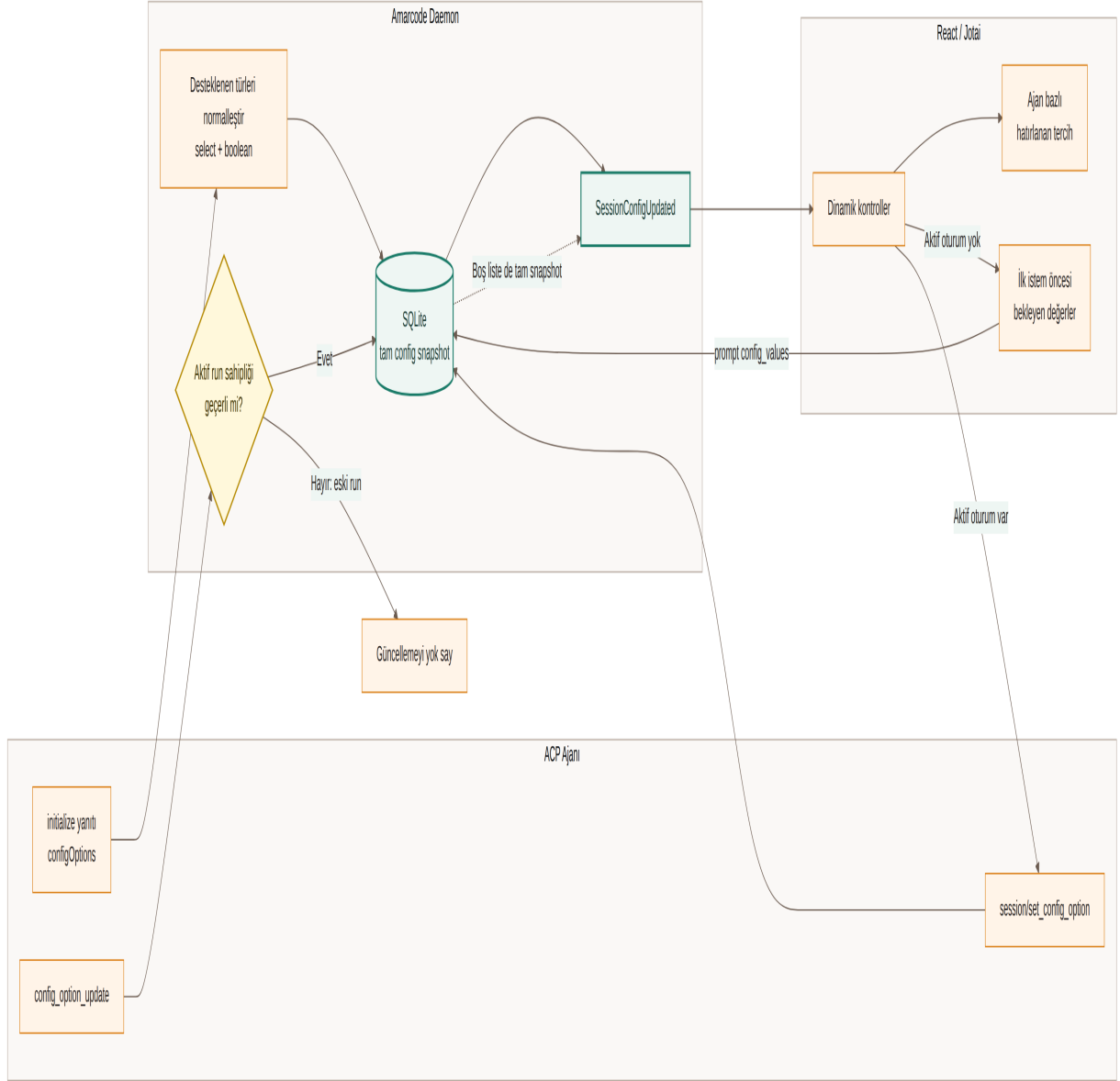
Şekil 7: Ajan registry ve kurulum akışı

Ek Şekil D.3 — Ajan manifestlerinin eşitlenmesi, dağıtım türlerinin işlenmesi, çalışma zamanı denetimi ve kurulum süreci.

Registry eşitlemesi en iyi çaba yaklaşımıyla uygulanır. Ağ bağlantısı yoksa daemon son başarılı checkout üzerinden çalışmaya devam eder. Manifestler bunx, uvx veya platforma özgü ikili dağıtımlara dönüştürülür. Bir ajanın katalogda bulunması ile yerel makinede kullanılabilir olması ayrı durumlar olarak değerlendirilir.

1.5 Ek E — Oturum Yapılandırmasının Eşitlenmesi

Ek Şekil E.1 — ACP ajanının bildirdiği yapılandırma seçenekleri ile daemon, SQLite ve React/Jotai durumu arasındaki çift yönlü veri akışı.



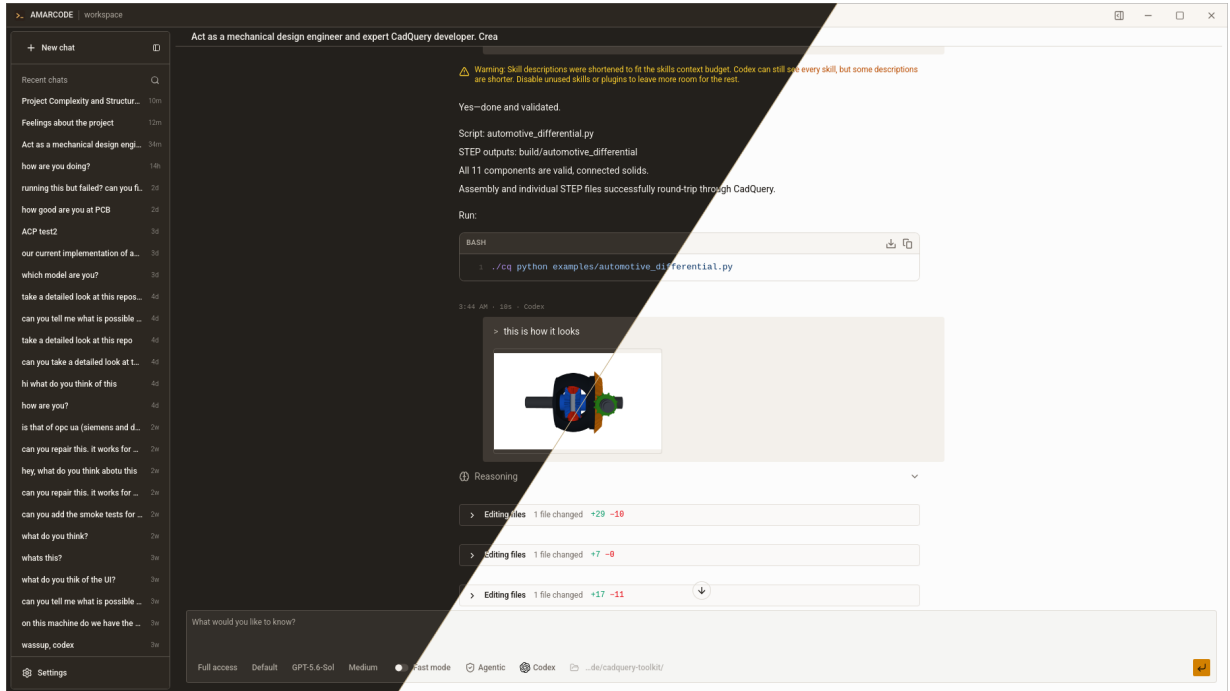
Şekil 8: Oturum yapılandırma eşitleme akışı

Ajanın bildirdiği seçim ve boolean seçenekleri genel arayüz kontrollerine dönüştürülür. İlk istemden önce seçilen değerler bekletilerek prompt isteğiyle gönderilir. Aktif oturum sırasında değiştirilen değerler session/set_config_option üzerinden ajana aktarılır. Yapılandırmanın tam anlık görüntüsü SQLite'ta saklanır; boş liste de önceki değerleri temizleyen geçerli bir anlık görüntü olarak kabul edilir. Ajan tarafından gönderilen güncellemeler yalnızca aktif çalışma sahipliği doğrulandıktan sonra arayüze uygulanır.

1.6 Ek F — Seçilmiş Uygulama Ekranları

Bu bölümdeki ekran görüntüleri uygulamanın 7 Eylül 2026 tarihindeki son durumunu göstermektedir. Görseller, ilgili özelliklerin rapor boyunca açıklanan geliştirme çalışmalarının sonucunu göstermesi amacıyla eklenmiştir; ekran görüntüsünün çekildiği tarih, özelliğin geliştirildiği staj günü olarak yorumlanmamalıdır.

1.6.1 Ek F.1 — Açık ve koyu tema



Şekil 9: Amarcode açık ve koyu tema

Ek Şekil F.1 — Amarcode'un açık ve koyu temalarının karşılaştırmalı görünümü.

1.6.2 Ek F.2 — Yeni sohbet ekranı

Ek Şekil F.2 — Çalışma alanı, ajan ve yapılandırma seçiminin ardından yeni görev başlatılan sade başlangıç ekranı.

1.6.3 Ek F.3 — Dosya ağacı ve kaynak görüntüleyici

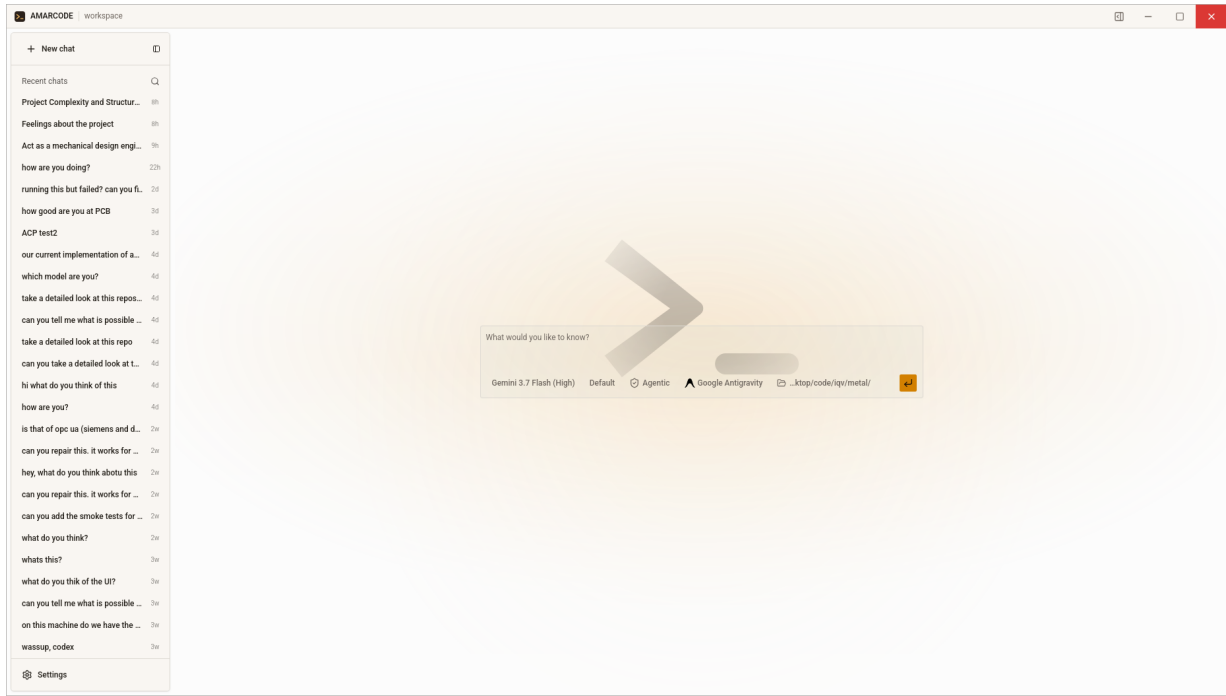
Ek Şekil F.3 — Aktif sohbetin yanında açılan çalışma alanı dosya ağacı ve kaynak kodu görünümü.

1.6.4 Ek F.4 — Diff inceleme

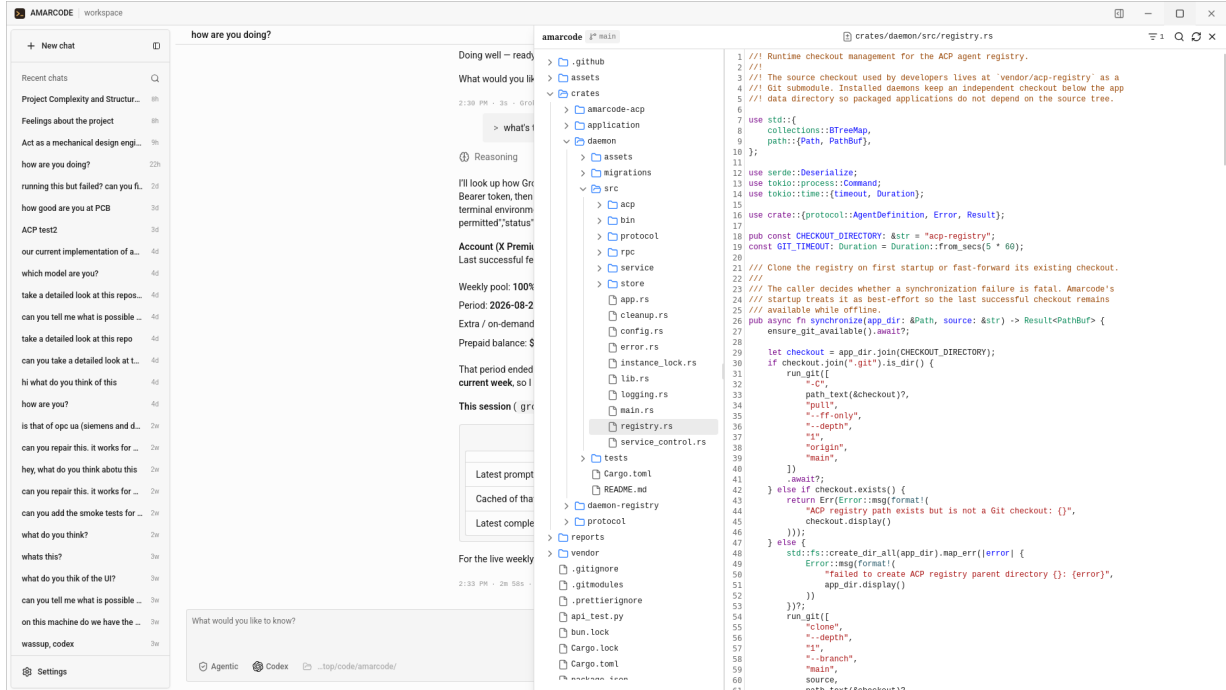
Ek Şekil F.4 — Ajan tarafından oluşturulan değişikliklerin eklenen ve silinen satırlar hâlinde incelenmesi.

1.6.5 Ek F.5 — Ajan seçimi

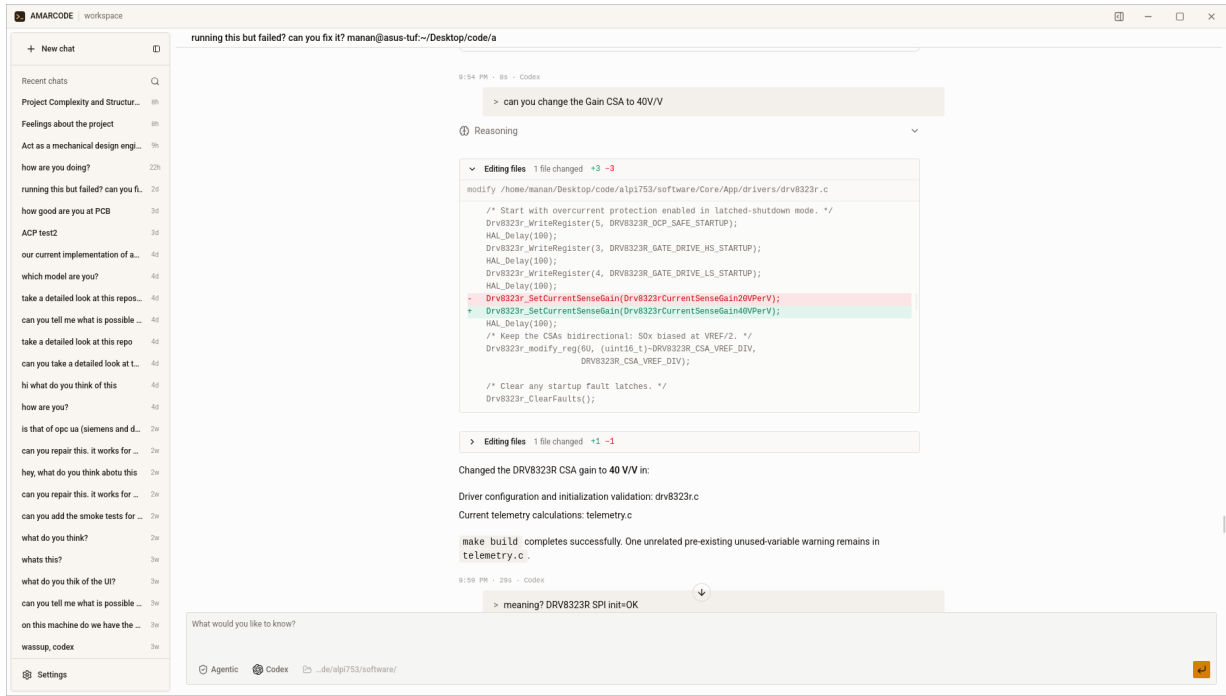
Ek Şekil F.5 — Registry tarafından sağlanan ajanlar ile kullanılabilirlik ve kurulum durumları.



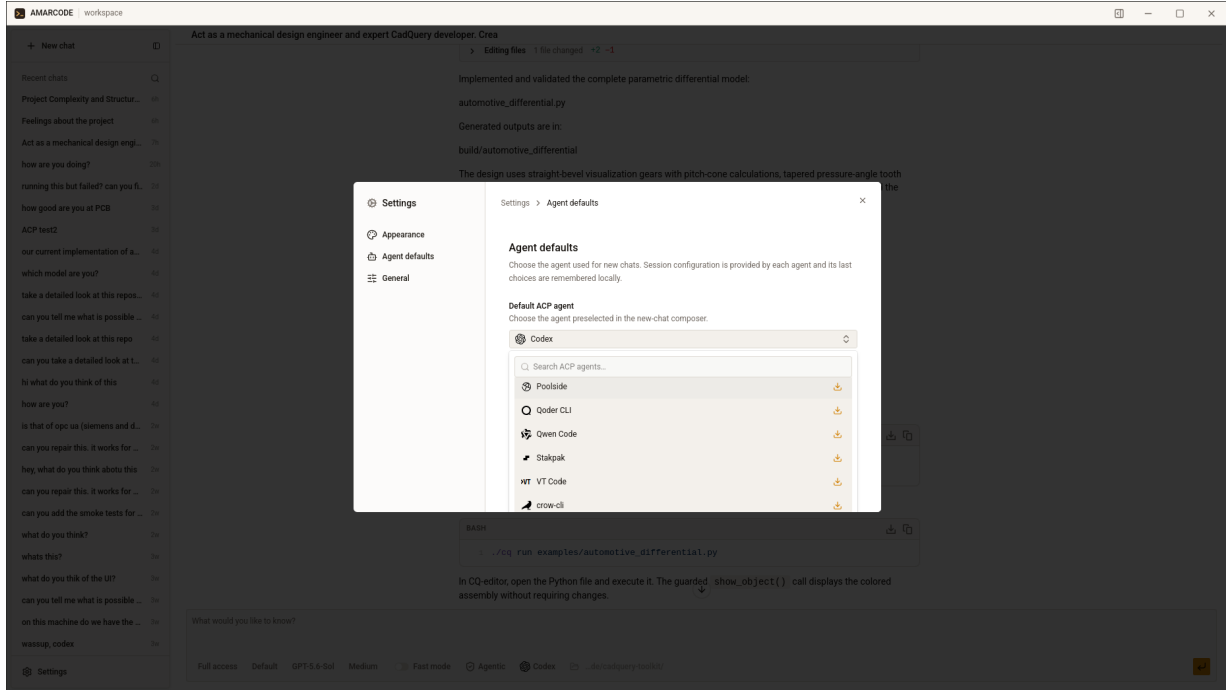
Şekil 10: Amarcode yeni sohbet ekranı



Şekil 11: Dosya ağacı ve kaynak görüntüleyici

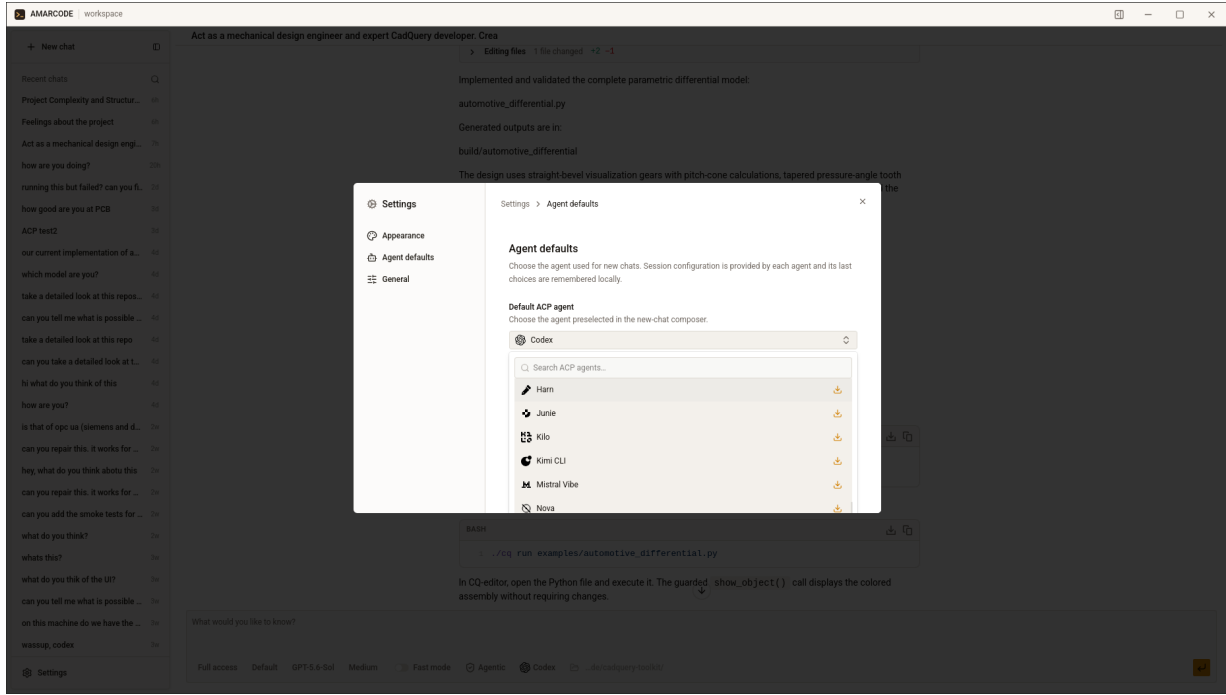


Şekil 12: Amarcode diff inceleme ekranı



Şekil 13: Amarcode ajan seçimi

1.6.6 Ek F.6 — Model ve yapılandırma seçenekleri



Şekil 14: Ajan yapılandırma seçenekleri

Ek Şekil F.6 — Seçilen ACP ajanının bildirdiği model ve oturum yapılandırma kontrolleri.

1.7 Ek G — İstemci-Daemon RPC Özeti

RPC yöntemi	Amaç
health	Daemon durumu, sürümü, protokol sürümü ve dinleme adresini döndürür
version	Daemon ve ortak protokol sürümünü bildirir
subscribe_events	Sohbet, çalışma veya oturum filtresiyle canlı olay akışına abone olur
list_agents	Registry'den elde edilen ajanları ve yerel kullanılabilirliklerini listeler
install_agent	Manifeste belirtilen dağıtım yöntemini kullanarak ajanı kurar
authenticate_agent	Kimlik doğrulaması gerektiren ajan için ACP doğrulama akışını başlatır
create_chat	Çalışma alanına bağlı kalıcı sohbet oluşturur
list_chats	İsteğe bağlı çalışma alanı filtresiyle sohbetleri getirir
get_chat	Sohbeti, mesajları ve mesaj parçalarını getirir
delete_chat	Sohbet ve ilişkili kalıcı kayıtları temizler
prompt	Kullanıcı istemini, ekleri ve başlangıç yapılandırmasını ajan çalışmasına aktarır
set_session_config_option	Aktif ACP oturumundaki bir yapılandırma değerini değiştirir
cancel	Sohbetteki aktif ajan dönüşünü iptal eder
respond_permission	Ajanın bekleyen izin isteğine cevap verir
respond_input	Ajanın kullanıcıdan istediği ek bilgiye cevap verir

RPC türlerinin güncel tanımları [crates/protocol/src/rpc.rs](https://github.com/amarcode/crates/protocol/src/rpc.rs) dosyasında bulunmaktadır.

1.8 Ek H — Canlı Olay Türleri

Olay	Arayüzdeki anlamı
chatUpdated runUpdated	Sohbet başlığı veya sohbet listesi değişmiştir Ajan çalışmasının başlangıç, çalışma, bitiş veya hata durumu değişmiştir
turnUpdated	Belirli kullanıcı isteminin dönüş durumu değişmiştir
contextRestoration	Önceki ACP oturumu veya kalıcı konuşma geçmişi geri yüklenmektedir
messageUpdated	Bir mesajın akış veya tamamlanma durumu değişmiştir
messagePartAdded	Metin, düşünme, araç veya diğer yapılandırılmış mesaj parçası eklenmiştir
approvalRequired questionRequired	Ajan bir işlem için kullanıcı izni beklemektedir Ajan görev için kullanıcıdan ek bilgi istemektedir
workspaceFilesChanged	Çalışma alanındaki bir veya daha fazla dosya değişmiştir
agentConnectionChanged	Ajan bağlantısı kurulmuş, kesilmiş veya hata vermiştir
agentAuthRequired	Seçilen ajan kimlik doğrulaması gerektirmektedir
sessionConfigUpdated	Oturuma ait yapılandırma seçenekleri güncellenmiştir

Olay türlerinin güncel tanımları [crates/protocol/src/events.rs](https://crates.io/protocol/src/events.rs) dosyasında bulunmaktadır.

1.9 Ek I — Test ve Doğrulama Matrisi

Alan	Doğrulama yaklaşımı	İlgili kaynak
Ön yüz canlı durumu	Mesaj parçaları, eski çalışma olayları ve sohbet geçişleri için durum testleri	daemon-events.test.ts
Oturum yapılandırması	Tam ve boş anlık görüntü, ajan değişimi ve bekleyen ilk istem değerleri	session-config.test.ts
Canlı sohbet	İyimser mesaj, akışlı içerik ve tamamlanma davranışları	live-chat.test.ts
İzin modu	İzin düzeyleri ve kullanıcı tercihleri	permission-mode.test.ts
ACP protokolü	Oturum, mesaj kimliği, iptal, araç ve terminal mesaj sıraları	protocol_transcript.rs
Daemon dikey dilimi	İstemden veritabanına, ACP'ye ve canlı olaya kadar uçtan uca akış	vertical_slice.rs
Ortak protokol	Üretilmiş TypeScript bağlarının Rust tanımlarıyla güncel kalması	generate-types.rs
Registry Worker	Kayıt API davranışı ve sürüm verisi	daemon-registry
Masaüstü dağıtımı	GitHub Actions üzerinde platform derleme ve paketlenme	.github/workflows

Projede tanımlanan temel doğrulama komutları şunlardır:

```
bun run app:test
bun run app:lint
bun run tsc
bun run worker:test
cargo test --workspace
cargo clippy --workspace --all-targets --all-features -- -D warnings
cargo fmt --all --check
```

1.10 Ek J — Git Tabanlı Geliştirme Özeti

İncelenen Git geçmişi 7 Ağustos 2026 ile 7 Eylül 2026 arasındaki **114 commit** üzerinden değerlendirilmiştir. Commit tarihleri günlük rapor başlıklarıyla bire bir eşleştirilmemiş, proje gelişiminin teknik sırası korunarak 18 staj gününe dağıtılmıştır.

Geliştirme dönemi	Öne çıkan çalışmalar
İlk prototip	İstem girişi, çalışma ekranı, TCP daemon ve test istemcisi
Arayüz-daemon birleşimi	RPC, iyimser mesajlar, olay tabanlı yükleme ve Jotaİ durum yönetimi
Güvenilirlik	Eş zamanlı istem kilitleri, eski olay koruması, abonelik ve yeniden bağlantı
Ortak protokol	Rust türlerinin merkezileştirilmesi, TypeScript üretimi ve sürüm el sıkışması
Daemon dağıtımı	Kullanıcı servisi, tek örnek kilidi, indirme, güncelleme, geri alma ve kaldırma
Çalışma alanı deneyimi	Dosya ağacı, arama, ekler, diff görüntüleyici ve dosya bağlantıları
ACP uyumluluğu	Typed ACP, iptal, araç çağrılar, terminal ve izin yönetimi
Registry sistemi	Ajan keşfi, manifest ayrıştırma, kullanılabilirlik ve uçtan uca kurulum
Son entegrasyon	Dinamik oturum seçenekleri, kimlik doğrulama, testler ve arayüz düzenlemeleri

Git geçmişi: [Amarcode commit listesi](#)

1.11 Ek K — Bilinen Sınırlamalar ve Gelecek Çalışmalar

- ACP adaptöründe oturum listeleme, silme ve tam kalıcı devam ettirme yetenekleri genişletilebilir.
- Görsel, ses ve ek MCP özellikleri desteklenen ajan yeteneklerine göre eklenebilir.
- Erişilebilirlik kontrolleri ve klavye ile tam gezinme için daha kapsamlı testler uygulanabilir.
- Windows, Linux ve macOS paketleri için daha geniş otomatik uçtan uca test matrisi hazırlanabilir.
- Daemon güncelleme ve bağlantı hataları için kullanıcıya sunulan tanılama bilgileri geliştirilebilir.
- Çok büyük çalışma alanlarında dosya ağacı, arama ve diff görünümü için performans ölçümleri yapılabilir.
- Wayland gibi platforma özgü pencere efektleri kontrollü özellik algılama ile ele alınabilir.
- Registry manifestleri için imza doğrulama ve daha ayrıntılı güven zinciri değerlendirilebilir.

1.12 Ek L — İlgili Belgeler

- [18 günlük raporların bulunduğu klasör](#)
- [Toplu kaynakça](#)
- [Amarcode README](#)
- [Daemon teknik belgesi](#)
- [ACP adaptörü teknik belgesi](#)